TUTORIAL

EC9590

NETWORK APPLICATION DESIGN

BANDARA H.G.T.D.

2022/E/048

SEMESTER 06

2026/02/14

PART A.

```
#include "mpi.h"
#include <stdio.h>

int main(argc,argv)
int argc;
char *argv[]; {
int    numtasks, rank, rc;

rc = MPI_Init(&argc,&argv);
if (rc != 0) {
  printf ("Error starting MPI program. Terminating.\n");
  MPI_Abort(MPI_COMM_WORLD, rc);
  }

MPI_Comm_size(MPI_COMM_WORLD,&numtasks);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
printf ("Number of tasks= %d My rank= %d\n", numtasks,rank);

/*******  do some work *******/

MPI_Finalize();
}
```

FIGURE 01: Code in part a

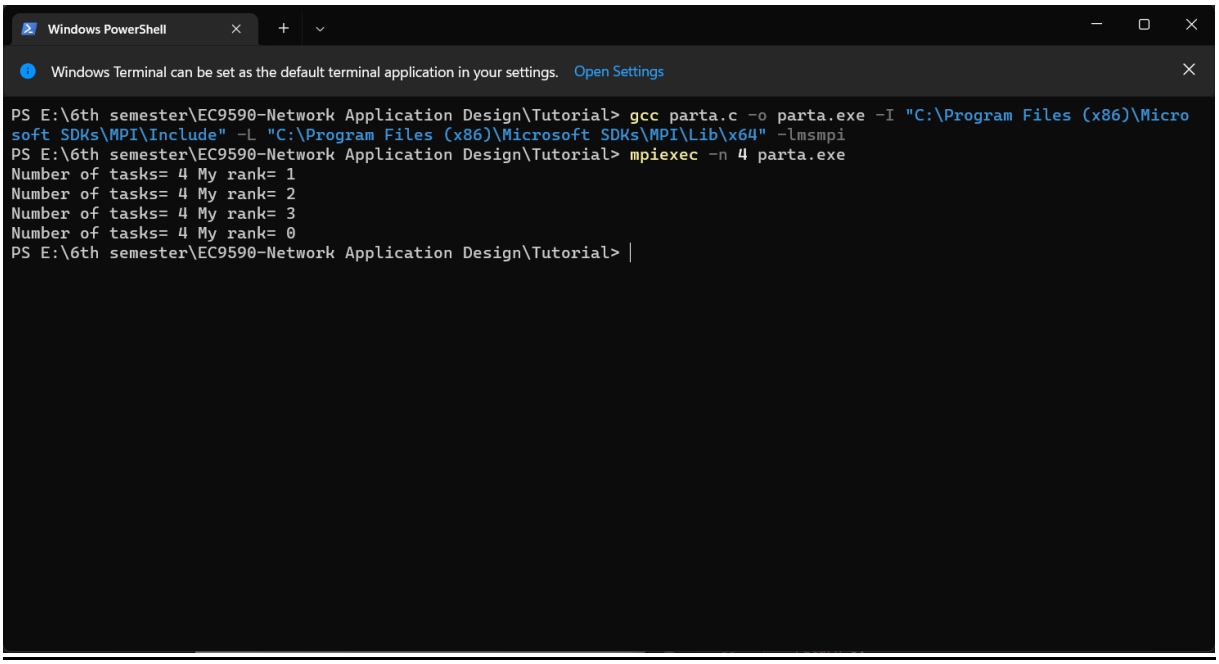

FIGURE 02: output in part a

What the Program Does :

This program sets up the MPI environment. It asks the system how many total processes are running and which specific "Rank" (ID number) the current process is assigned. Finally, it prints this information so you can see that multiple processes are running at the same time .

3. MPI Functions Used

- **MPI_Init**: Initializes the MPI execution environment. This function must be called in every MPI program before any other MPI functions.

- **MPI_Comm_size**: Determines the number of processes in the group associated with a communicator (usually MPI_COMM_WORLD).

- **MPI_Comm_rank**: Determines the rank (ID) of the calling process within the communicator. Each process gets a unique integer rank between 0 and (number of processors - 1).

- **MPI_Abort**: Terminates all MPI processes associated with the communicator. It is used here to stop the program if an error occurs during startup.

- **MPI_Finalize**: Terminates the MPI execution environment. This should be the last MPI routine called in every MPI program.

PART B.

```
#include "mpi.h"
#include <stdio.h>

int main(argc,argv)
int argc;
char *argv[]; {
int    numtasks, rank, rc;

rc = MPI_Init(&argc,&argv);
if (rc != 0) {
  printf ("Error starting MPI program. Terminating.\n");
  MPI_Abort(MPI_COMM_WORLD, rc);
  }

MPI_Comm_size(MPI_COMM_WORLD,&numtasks);
MPI_Comm_rank(MPI_COMM_WORLD,&rank);
printf ("Number of tasks= %d My rank= %d\n", numtasks,rank);

/*******  do some work *******/

MPI_Finalize();
}
```

FIGURE 03: Code in part b

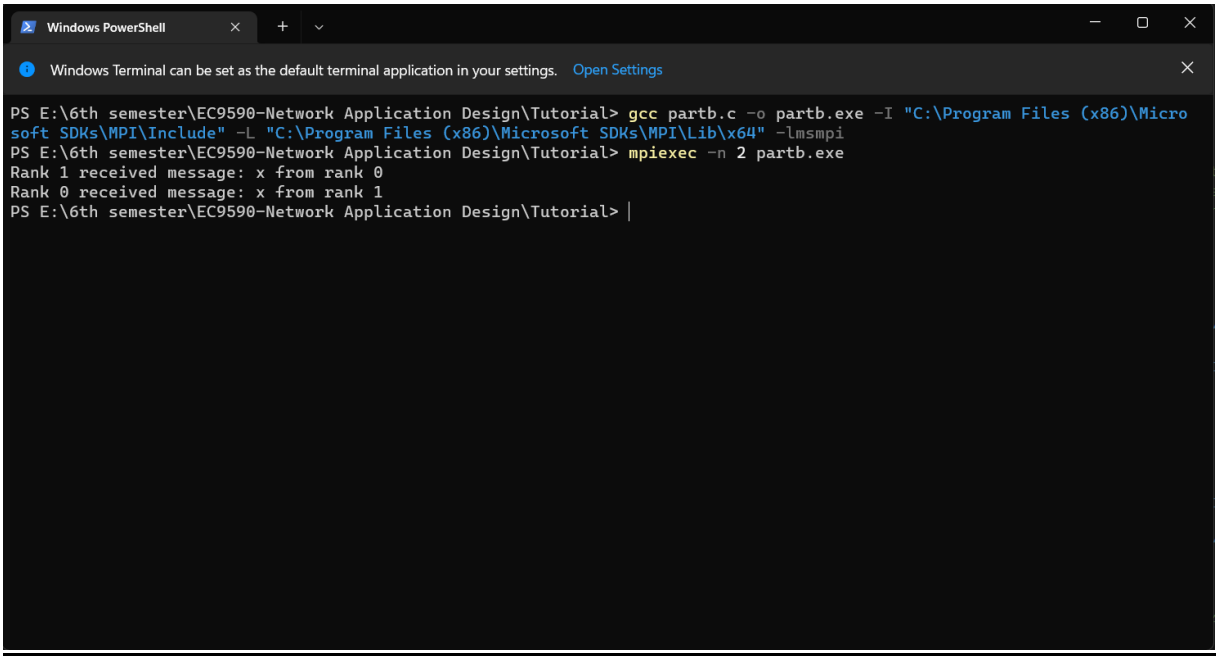

FIGURE 04: output of part b

What the Program Does:

This program creates a "ping-pong" exchange between two processes:

- Rank 0 sets a destination of 1. It sends a message to Rank 1 and then waits to receive a message back .

- Rank 1 sets a destination of 0. It waits to receive a message from Rank 0 and then sends a message back .

MPI Functions Used:

- **MPI_Send:** A basic blocking send operation. The routine returns only after the application buffer in the sending task is free for reuse.

- **MPI_Recv:** Receives a message and blocks (waits) until the requested data is available in the application buffer.

- **MPI_Init**: Initializes the MPI execution environment. This function must be called in every MPI program before any other MPI functions.

- **MPI_Comm_size**: Determines the number of processes in the group associated with a communicator (usually MPI_COMM_WORLD).

- **MPI_Comm_rank**: Determines the rank (ID) of the calling process within the communicator. Each process gets a unique integer rank between 0 and (number of processors - 1).

- **MPI_Finalize**: Terminates the MPI execution environment. This should be the last MPI routine called in every MPI program.

PARTC:

```
#include "mpi.h"
#include <stdio.h>

int main(argc,argv)
int argc;
char *argv[];  {
int numtasks, rank, dest, source, rc, tag=1;
char inmsg, outmsg='x';
MPI_Status Stat;

MPI_Init(&argc,&argv);
MPI_Comm_size(MPI_COMM_WORLD, &numtasks);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if (rank == 0) {
  dest = 1;
  source = 1;
  rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
  rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
  }

else if (rank == 1) {
  dest = 0;
  source = 0;
  rc = MPI_Recv(&inmsg, 1, MPI_CHAR, source, tag, MPI_COMM_WORLD, &Stat);
  rc = MPI_Send(&outmsg, 1, MPI_CHAR, dest, tag, MPI_COMM_WORLD);
  }

MPI_Finalize();
}
```

FIGURE 05: Code in part c

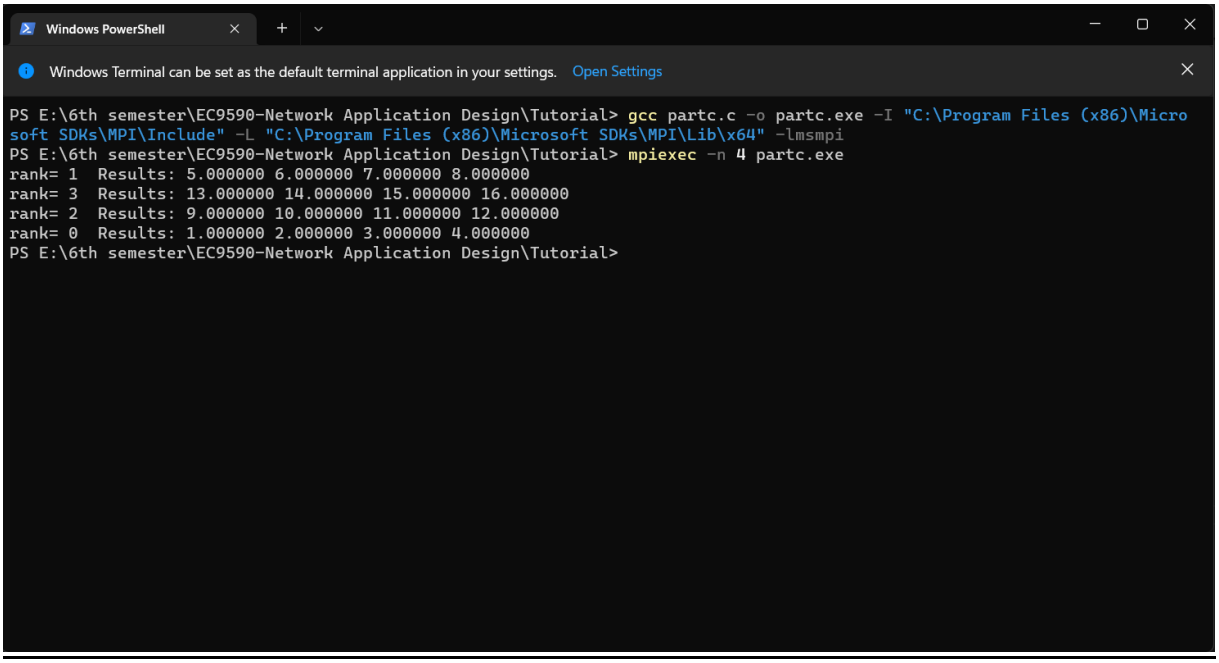

FIGURE 06: Output of part c

What the Program Does:

The program defines a 4 x 4 matrix of numbers. It checks if exactly 4 processors are running. If so, **Rank 1** (the source) takes the matrix and splits it up, sending one row to every process (including itself). Each process then prints the row of numbers it received.

MPI Functions Used

- MPI_Scatter: This function is used to split data from one process (the source) and send distinct parts of it to all other processes in the group.

- **MPI_Init**: Initializes the MPI execution environment. This function must be called in every MPI program before any other MPI functions.

- **MPI_Comm_size**: Determines the number of processes in the group associated with a communicator (usually MPI_COMM_WORLD).

- **MPI_Comm_rank**: Determines the rank (ID) of the calling process within the communicator. Each process gets a unique integer rank between 0 and (number of processors - 1).

- **MPI_Finalize**: Terminates the MPI execution environment. This should be the last MPI routine called in every MPI program.